

SKILL EXPERIMENT 18

Name: K ANSHU

Roll No: 2520030440

Section: 8

Title

Implementation of Signal Handling and Background Job Recovery

Aim

To analyze signal delivery, understand signal masks, review process groups, explore terminal signals, and test signal behavior.

To implement the resumption of stopped jobs, send continue signals, update job status, maintain the job table, verify execution, and test background recovery.

Theory

Signals are used by the operating system to notify processes about events or changes in their execution state. Signal delivery allows a process to respond to events such as termination, stopping, or continuation.

Signal masks are used to control which signals are temporarily blocked from being delivered to a process. Process groups allow related processes to be managed together, while terminal-generated signals can affect the processes associated with the terminal.

A stopped job can be resumed by sending an appropriate continue signal. After the job resumes, its state is updated in the job table. Maintaining accurate job information allows the shell to verify execution and correctly track background job recovery.

Algorithm

Part A – Signal Handling

1. Start the program.
2. Initialize the required signal handling mechanism.
3. Identify the signals used by the process.
4. Analyze how signals are delivered to the process.
5. Configure the required signal mask.

6. Block or allow selected signals as required.
7. Identify the process group associated with the process.
8. Observe signals generated through the terminal.
9. Handle the received signals.
10. Display the signal behavior and process state.
11. Test the signal handling operation.
12. Stop the signal analysis operation.

Part B – Background Job Recovery

1. Start the program.
2. Initialize the job table.
3. Identify a stopped background job.
4. Retrieve the job ID and process group information.
5. Send a continue signal to the stopped job.
6. Resume execution of the selected job.
7. Update the job status from stopped to running.
8. Maintain the updated information in the job table.
9. Monitor the resumed job.
10. Verify that the job continues execution.
11. Update the job state when its execution changes.
12. Test background recovery with the selected job.
13. Display the final job status.
14. Stop the program.

CODE

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
#include <signal.h>
```

```
#include <sys/wait.h>
```

```
typedef struct {
```

```
    int job_id;
```

```
    pid_t pid;
```

```
    char state[20];
```

```
} Job;
```

```
int main() {  
  
    Job job;  
  
    pid_t pid;  
  
    int status;  
  
  
    job.job_id = 1;  
  
  
    pid = fork();  
  
  
    if (pid == -1) {  
        perror("fork");  
        return 1;  
    }  
  
  
    if (pid == 0) {  
        printf("Job %d started | PID = %d | State = Running\n",  
            job.job_id, getpid());  
  
  
        printf("Job is stopping itself using SIGSTOP...\n");  
        raise(SIGSTOP);  
  
  
        printf("Job resumed successfully using SIGCONT.\n");  
        printf("Job is running again...\n");  
    }  
}
```

```
    sleep(2);

    printf("Job completed.\n");

    return 0;
}

job.pid = pid;

sprintf(job.state, "Running");

sleep(1);

waitpid(pid, &status, WUNTRACED);

if (WIFSTOPPED(status)) {

    sprintf(job.state, "Stopped");

    printf("\nJob Table:\n");

    printf("Job ID: %d | PID: %d | State: %s\n",

        job.job_id, job.pid, job.state);

    printf("\nSending SIGCONT to resume Job %d...\n",

        job.job_id);

    kill(job.pid, SIGCONT);

    sprintf(job.state, "Running");
```

```
    printf("Job Status Updated: %s\n", job.state);  
}  
  
waitpid(pid, &status, 0);  
  
sprintf(job.state, "Completed");  
  
printf("\nFinal Job Table:\n");  
printf("Job ID: %d | PID: %d | State: %s\n",  
    job.job_id, job.pid, job.state);  
  
printf("\nBackground job recovery completed successfully.\n");  
  
return 0;  
}
```

COMPILTION

```
hp@DESKTOP-35T9S8P:~$ nano exp3.c
hp@DESKTOP-35T9S8P:~$
hp@DESKTOP-35T9S8P:~$ gcc exp3.c -o exp3
hp@DESKTOP-35T9S8P:~$ ./exp3
Job 1 started | PID = 679 | State = Running
Job is stopping itself using SIGSTOP...

Job Table:
Job ID: 1 | PID: 679 | State: Stopped

Sending SIGCONT to resume Job 1...
Job Status Updated: Running
Job resumed successfully using SIGCONT.
Job is running again...
Job completed.

Final Job Table:
Job ID: 1 | PID: 679 | State: Completed

Background job recovery completed successfully.
hp@DESKTOP-35T9S8P:~$ |
```

Result

Signal delivery, signal masks, process groups, and terminal signal behavior were analyzed successfully. Stopped background jobs were resumed using continue signals, their status was updated in the job table, and background recovery was verified successfully.